



United International University
Department of Computer Science and Engineering
CSE 1115: Object-Oriented Programming Spring 2026
Mid Term Total Marks: 30 Time: 90 minutes

Any examinee found adopting unfair means will be expelled
from the course as per UIU disciplinary rules.

Answer all the following questions. The numbers on the right denote their marks.

1. A financial company is developing a Java-based **asset assignment system** for its traders. The system consists of two classes: **Asset** and **Trader**.

An object of the **Asset** class represents a financial asset and contains the following attributes:

- **assetType**: the type of asset, such as "Stock" or "Bond"
- **price**: a value of type **float**
- **id**: an automatically generated identifier
- **tradersName**: the name of the trader to whom the asset is assigned

Only **assetType** and **price** are provided at the time of object creation. The **id** starts from 1001 and increases by 1 for each new object. The **tradersName** is initially an empty string. The **price** must not be accessed directly from outside the class; appropriate accessor and/or mutator methods must be provided.

An object of the **Trader** class represents a trader in the company. Each trader has the following attributes:

- **name**
- **department**
- **aum** (assets under management), of type **float**

The **name** and **department** are set during object creation. If the department is not provided, it should default to "Bond". The value of **aum** is initially 0.0 and increases as assets are assigned.

The **Trader** class contains a method **assign(Asset asset)**. This method assigns an asset to a trader only if the trader's department matches the asset's type. If the assignment is successful:

- the trader's **aum** increases by the asset's price, and
- the asset's **tradersName** is updated accordingly.

If the departments do not match, the method should print **Department Mismatch**.

Answer the following:

- (a) Define the **Asset** class, including the necessary attributes, constructor, and methods. (5)
- (b) Define the **Trader** class, including the necessary attributes, constructors, and method. (5)

If both classes are implemented correctly, the following main method should produce the output shown below.

Main.java

```
1 class Main {
2     public static void main(String[] args) {
3         Trader bond_trader = new Trader("Borsha");
4         Trader stock_trader = new Trader("Shondha", "Stock");
5
6         Asset stock = new Asset("Stock", 7000);
7
8         bond_trader.assign(stock);
9         stock_trader.assign(stock);
10
11        System.out.println("Name: " + stock_trader.name + ", AUM: " +
12                               stock_trader.aum);
13        System.out.println("Asset ID: " + stock.id + ", Trader: " +
14                               stock.tradersName);
15    }
16 }
```

Output

```
Department Mismatch
Name: Shondha, AUM: 7000.0
Asset ID: 1001, Trader: Shondha
```

2. Complete the missing parts of the following Java program so that it produces the expected output. Analyze the given code and infer the necessary variable declarations, constructor, and method implementations based on how they are used.

(7)

The program consists of the following files.

Animal.java

```
1 class Animal {
2
3     // TODO: Declare necessary variables
4     // TODO: Implement constructor
5
6
7     void makeSound() {
8         System.out.println("Animal makes a sound");
9     }
10
11     // TODO: Implement method
12 }
```

Dog.java

```
1 class Dog extends Animal {
2     Dog(String name, int age) {
3         super(name, age);
4     }
5
6     // TODO: Override method
7
8     void showDetails() {
9         System.out.println(super.getInfo());
10        System.out.println("This is a dog.");
11    }
12 }
```

Main.java

```
1 public class Main {
2     public static void main(String[] args) {
3         Dog d = new Dog("Buddy", 3);
4         d.makeSound();
5         d.showDetails();
6     }
7 }
```

Output

```
Buddy barks
Name: Buddy, Age: 3
This is a dog.
```

3. Trace the following Java program and write the exact output.

(8)

The program consists of the following files.

Test.java

```
1 class Test {
2     static int count = 0;
3     int id = 1;
4
5     static {
6         count += 2;
7         System.out.println("Static Block");
8     }
9
10    {
11        count += 3;
12        id += count;
13        System.out.println("Instance Block");
14    }
15
16    Test() {
17        count += 5;
18        id += 2;
19        System.out.println("Constructor");
20    }
21 }
```

Main.java

```
1 class Main {
2     public static void main(String[] args) {
3         Test t1 = new Test();
4         Test t2 = new Test();
5         Test t3 = t1;
6         t2.count += 10;
7         t3.id += 5;
8
9         System.out.println("t1-count = " + t1.count + ", t1-id = " +
10            t1.id);
11        System.out.println("t2-count = " + t2.count + ", t2-id = " +
12            t2.id);
13        System.out.println("t3-count = " + t3.count + ", t3-id = " +
14            t3.id);
15    }
16 }
```

4. Consider the following Java program. The program consists of the following files. Complete the main() method so that the program produces the following output. No print statements are allowed in the main() method. All output must come exclusively from the solveProblem(Employee[] employeeList) method.

Employee.java

```
1 class Employee {
2     String problem;
3
4     void assignProblem(String problem) {
5         this.problem = problem;
6     }
7 }
```

Manager.java

```
1 class Manager extends Employee {
2     void solveProblem() {
3         System.out.println("Manager is solving " + this.problem);
4     }
5 }
```

Engineer.java

```
1 class Engineer extends Employee {
2     void solveProblem() {
3         System.out.println("Engineer is solving " + this.problem);
4     }
5 }
```

Driver.java

```
1 public class Driver {
2     static void solveProblem(Employee[] employeeList) {
3         for (Employee emp : employeeList) {
4             if (emp instanceof Manager) {
5                 ((Manager) emp).solveProblem();
6             } else if (emp instanceof Engineer) {
7                 ((Engineer) emp).solveProblem();
8             }
9         }
10    }
11
12    public static void main(String[] args) {
13        // Complete the rest of the main method.
14        // You may use only the classes and methods already given.
15    }
16 }
```

Output

```
Manager is solving Budget planning issue
Engineer is solving Server bug
```